UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 05

Bit Manipulation

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Objetivos de la sesión **2**

2 Secuencia de clase **2**
2.1 Apertura (5 min) 2
2.2 Visualización: `bitset` y literales (10 min) 2
2.3 Operadores bit a bit (15 min) 3
2.4 Ejercicios en clase (10 min) 3

3 FAQs **4**

4 Soluciones a los ejercicios **4**
4.1 Desplazamientos 4
4.2 NOT, AND, OR y XOR 5
4.3 Combinando operadores 6

5 Demo en vivo sugerida **6**

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Leer y escribir literales binarios (`0b`) y hexadecimales (`0x`) en C++.
2. Usar `bitset<N>` para visualizar la representación binaria de un valor.
3. Calcular el resultado de `«`, `»`, `~`, `&`, `|` y `^` sobre valores de 8 bits a mano.
4. Identificar los tres patrones especiales (todo ceros, todo unos, máscara de un bit) y reconocerlos en código.
5. Evaluar expresiones que combinen varios operadores de bits paso a paso.

50 minutos. Sugerido: 5 min apertura + 10 min visualización y literales + 10 min patrones especiales + 15 min operadores lógicos + 10 min ejercicios. La sesión 06 es práctica en el cuaderno; estos ejercicios son en papel.

Secuencia de clase

- **Apertura (5 min)**

  Arranca con esta pregunta al grupo:

> *"¿Cuántas operaciones necesita el procesador para calcular `x * 8`?"*
>
> La mayoría dirá "una multiplicación". La respuesta real: si usas `x « 3`, el procesador lo resuelve con **un solo ciclo de reloj**. Una multiplicación normal tarda entre 3 y 5 ciclos. Ese ahorro, repetido millones de veces, importa.
>
> Sigue con: *"Hoy vamos a aprender a hablarle al procesador en su idioma natural: bits."*

- **Visualización: `bitset` y literales (10 min)**

  Muestra en el cuaderno (o proyecta) la celda de código con `bitset`:

```
int x = 30;
cout << bin8(x);    // 00011110
```

Pregunta al grupo: *"¿Por qué `00011110`?"* Deja que los alumnos cuenten las potencias de 2 en voz alta: $16 + 8 + 4 + 2 = 30$.

> Muestra `0b00011110`, `0x1E` y `30` imprimiendo exactamente lo mismo. El punto: la base es solo presentación. El compilador guarda los mismos bits sin importar cómo escribiste el literal.
>
> Pregunta de sondeo: *"¿Qué imprimiría `cout « 0b0101`?"* — Respuesta: 5. No

imprime “0101”; para eso se necesita `bitset`.

Después introduce los patrones especiales. La tabla de `1 « k` funciona bien en el cuaderno porque los alumnos pueden ejecutarla celda por celda y ver el bit moverse.

- ## Operadores bit a bit (15 min)

  Presenta NOT, AND, OR y XOR con sus tablas de verdad. Para cada uno:

1. Tabla de verdad de 1 bit.
2. Ejemplo con 8 bits (alinear operandos en columnas en el pizarrón).
3. El “uso típico” — para qué sirve en la práctica.

Varios alumnos confundirán `&` (AND bit a bit) con `&&` (AND lógico). El AND lógico trata todo valor distinto de cero como `true`; el AND bit a bit opera posición a posición. Ejemplo que los distingue:

	`a = 2`	`b = 1`	Resultado
`a && b`	(true)	(true)	`1` (true)
`a & b`	`10`	`01`	`0`

El mismo par de valores da resultados completamente distintos.

Si el grupo lleva bien el ritmo, menciona el truco: aplicar XOR con la misma clave dos veces recupera el original (`(x ^ k) ^ k = x`). Es la base del cifrado Vernam (one-time pad), el único sistema de cifrado matemáticamente irrompible. Funciona bien como anécdota de cierre.

- ## Ejercicios en clase (10 min)

  Los ejercicios de la sección 7 de las notas del alumno. Recomendado:

- Los ejercicios 1, 4 y 7a en clase (desplazamientos + tabla AND/OR + expresión combinada).
- Los demás quedan como tarea para reforzar antes de la sesión 06 (práctica en cuaderno).

Este ejercicio es una pista hacia la sesión 06. `n & (n-1)` apaga el bit encendido de menor peso. Si `n` es potencia de 2, solo tiene un bit encendido, así que el resultado es cero. Es el corazón del algoritmo para contar bits activos (popcount eficiente) que verán en la sesión 06.

FAQs

En complemento a 2, `~x = -(x+1)`. Intuitivamente: `x + ~x = -1` (todos los bits encendidos), así que `~x = -1 - x = -(x+1)`.

Verificación rápida: `~0 = -1` (todo 1s), `~(-1) = 0` (todo 0s).

En hardware moderno la diferencia es menor que antes (los procesadores tienen multiplicadores muy optimizados), pero el compilador **ya** convierte `x * 8` en `x << 3` automáticamente cuando puede. La razón real para aprender desplazamientos: leer código de sistemas y programación competitiva donde el idioma son bits, no aritmética.

El estándar C++ (hasta C++19) dice que el comportamiento de `>>` sobre negativos es *implementation-defined*. En la práctica, g++ y clang hacen extensión de signo (aritmética), así que `-8 >> 1 = -4`. Pero el código correcto usa `unsigned int` si necesita desplazar a la derecha de forma portable.

En complemento a 2, -1 se almacena como $2^n - 1$. Para 32 bits: $2^{32} - 1 = 4{,}294{,}967{,}295 =$ `0xFFFFFFFF` = todo 1s.

Forma alternativa de verlo: `0` tiene todo en 0. Le restamos 1 y como no podemos (prestamos de un bit que no existe), todos los bits se encadenan y quedan en 1.

Soluciones a los ejercicios

• Desplazamientos

a) `7 << 2` $= 7 \times 4 = 28$; binario: `00011100`.

b) `100 >> 2` $= \lfloor 100/4 \rfloor = 25$; binario: `00011001`.

c) `1 << 5` $= 32$; binario: `00100000`.

d) `96 >> 3` $= \lfloor 96/8 \rfloor = 12$; binario: `00001100`.

$48/3 = 16 = 2^4$, así que cuatro desplazamientos a la izquierda. Verificación: `3 << 4` $= 3 \times 16 = 48$. ✓

Si `x` tiene a lo más 8 bits, desplazarlo 8 posiciones expulsa todos sus bits; el resultado es `0`. (Técnicamente, si `x` es `int` de 32 bits, los 24 bits altos son 0, y todos los bits de `x` "caen" fuera por la derecha.)

• NOT, AND, OR y XOR

a) `17 & 12`:
`00010001 & 00001100 = 00000000 = 0`. No comparten ningún bit encendido.

b) `13 | 12`:
`00001101 | 00001100 = 00001101 = 13`. Los bits de 12 ya están en 13, así que OR no aporta nada nuevo.

c) `17 ^ 12`:
`00010001 ^ 00001100 = 00011101 = 29`.

d) `~12 = -(12+1) = -13`. En 32 bits: `11111111111111111111111111110011`.

a) `x & 0b00001111 = 0b00000011 = 3`. AND con `0x0F` **conserva solo los 4 bits bajos** (nibble bajo) y apaga los 4 altos. Operación clásica para extraer el nibble bajo.

b) `x | 0b00001111 = 0b10111111 = 191`. OR con `0x0F` **enciende los 4 bits bajos** sin tocar los demás.

c) `x ^ 0b11111111 = 0b01001100 = 76`. XOR con `0xFF` **invierte todos los bits**, equivalente a NOT para los 8 bits bajos. Notar que difiere de `~x` en que este solo actúa sobre 8 bits, no los 32.

a	b	a & b	a | b
11001100	10101010	10001000	11101110
11110000	00001111	00000000	11111111
10101010	10101010	10101010	10101010

La segunda fila es interesante: cuando los patrones son **complementos** entre sí, AND da todo 0s y OR da todo 1s. La tercera fila: AND y OR con sí mismo devuelve el original.

- **Combinando operadores**

Paso a paso:

1. `~7 = ...11111000` (apaga los bits 0, 1, 2)
2. `| (1≪1) = ...11111010` (enciende el bit 1)
3. `| (1≪3) = ...11111010` (bit 3 ya estaba)
4. `| (1≪5) = ...11111010` (bit 5 ya estaba)
5. `29 = 0b00011101`
6. AND: `0b00011101 & 0b11111010 = 0b00011000 = 24`

1. `13 = 0b00001101`
2. `13 ≫ 2 = 0b00000011 = 3`
3. `~3 = ...11111100`
4. `39 = 0b00100111`
5. `~3 & 39 = 0b00100100 = 36`

Por las propiedades del XOR: `y ^ y = 0`, y `x ^ 0 = x`. Resultado: siempre `x`, sin importar `y`. Este truco es la base del *XOR swap* (intercambiar dos variables sin una temporal) que verán en la sesión 06.

`12 = 0b00001100.  11 = 0b00001011.`

`12 & 11 = 0b00001000 = 8.`

El resultado apaga el bit encendido de **menor peso** de `n` (el bit 2 en este caso). Si `n` es potencia de 2 (un solo bit encendido), el resultado es 0. Esta es la prueba más eficiente para saber si un número es potencia de 2: `n > 0 && (n & (n-1)) == 0`.

Demo en vivo sugerida

Esta demo conecta la sesión 04 (ASCII) con la sesión 05 (bits):

```
char c = 'A';                // 0x41 = 0100 0001
char lower = c ^ 0x20;      // 0x61 = 0110 0001  = 'a'
char back  = lower ^ 0x20; // 0x41 = 0100 0001  = 'A'
```

El XOR con `0x20` invierte el bit 5, que es exactamente el bit que distingue mayúsculas de minúsculas en ASCII. Aplicarlo dos veces devuelve la letra original.

Punto de discusión: ¿funciona igual con `|` o `&`? No: OR solo sube, AND solo baja. XOR es el único que *alterna*.

Si la sesión 05 va bien y sobra tiempo, abre el cuaderno y ejecuta la celda de los patrones especiales (`1 << k` del 0 al 7). Ver el bit "caminar" hacia la izquierda en la terminal suele generar buena discusión sobre lo que físicamente representa cada bit en la memoria.